

Руководство MBS-fast

9 июня 2026

Оглавление

1 Сборка	1
1.1 Требования	1
1.2 CPU	2
1.3 GPU	2
1.4 EPYC Zen5, AVX-512 и AVX2	3
2 Быстрый старт	3
3 Физическая модель	3
3.1 Общий pipeline	3
3.2 Jones матрицы лучей	4
3.3 Диады и поворот в базис рассеяния	4
3.4 Дифракционный интеграл Kirchhoff	4
3.5 Когерентная и некогерентная сумма	5
3.6 Усреднение ориентаций и $-\text{pole}$	5
3.7 Beca scattering grid	5
4 Частицы и размер	5
5 Ориентационные сетки	6
6 Сетка рассеяния	6
7 GPU и multi-GPU	7
7.1 За счет чего GPU быстрее	7
7.2 Atomic и no-atomic режимы	7
7.3 Multi-size и multi-GPU	8
8 Все флаги	8
8.1 Метод, частица, физические параметры	8
8.2 Ориентации	8
8.3 Сетка, ускорение, cutoffs	9
8.4 Output, diagnostics, legacy	9
9 Переменные окружения	10
10 Выходные файлы	10

MBS-fast считает рассеяние света несферическими ледяными частицами: сначала трассируются геометрические лучи, затем для выходных пучков считается дифракция в приближении Physical Optics. Код поддерживает CPU-расчеты через OpenMP/MPI и CUDA-ускорение дифракции на GPU.

1 Сборка

1.1 Требования

Компонент	Для чего нужен	Комментарий
GCC >= 9 или Clang >= 14	CPU и host-часть CUDA	На ЕРҮС обычно используется GCC
OpenMP	Многопоточность CPU	Для GCC линкуется libgomp
MPI	CPU split build	cpu/Makefile использует mpicxx, если он есть
CUDA toolkit	GPU split build	Нужны nvcc, libcudart, libcufft
NVIDIA driver	GPU запуск	Драйвер должен поддерживать собранный sm_XX

Рекомендуемый путь сборки - split build:

```
make -C cpu -j          # CPU MPI/OpenMP
make -C gpu -j          # GPU float_fast по умолчанию
```

Объектные файлы CPU лежат в cpu/build/, CUDA - в gpu/build/. Общая физика и CLI остаются в src/, поэтому CPU и GPU версии не расходятся в разные кодовые базы.

1.2 CPU

```
# MPI + OpenMP CPU binary
```

```
make -C cpu -j
```

```
# Один MPI rank, 64 OpenMP потока
```

```
OMP_NUM_THREADS=64 cpu/bin/mbs_po_mpi --po -p 1 125.9 78.09 \
  --ri 1.3116 0 -w 0.532 -n 12 --oldauto 2 \
  --grid 0 180 600 180 --threads 64 --close -o out_cpu
```

```
# Четыре MPI rank, по 16 потоков на rank
```

```
mpirun -np 4 cpu/bin/mbs_po_mpi --po -p 1 125.9 78.09 \
  --ri 1.3116 0 -w 0.532 -n 12 --oldauto 2 \
  --grid 0 180 600 180 --threads 16 --close -o out_cpu_mpi
```

Debug/help сборка:

```
make -C cpu debug
```

```
cpu/bin/mbs_po_mpi_debug --help-debug
```

1.3 GPU

```
# GPU binary по умолчанию: float + fast math
```

```
PATH=/usr/local/cuda/bin:$PATH make -C gpu -j
```

```
# Явные варианты
```

```
make -C gpu float -j      # gpu/bin/mbs_po_gpu_float
```

```
make -C gpu float_fast -j # gpu/bin/mbs_po_gpu_float_fast
```

```
make -C gpu double_fast -j # gpu/bin/mbs_po_gpu_double_fast
```

Target	Binary	CUDA точность	Fast math	Когда ис
make -C gpu	gpu/bin/mbs_po_gpu_float_fast	FP32	да	Быстрые
make -C gpu float	gpu/bin/mbs_po_gpu_float	FP32	нет	Проверка
make -C gpu double_fast	gpu/bin/mbs_po_gpu_double_fast	FP64	да	Контроль
make -C gpu double_debug	gpu/bin/mbs_po_gpu_double_debug	FP64	да	--help-c

В split GPU build CUDA backend включен по умолчанию. Флаг - -gpu можно писать, но он не обязателен. Флаг - -cpu принудительно запускает CPU backend внутри GPU-capable бинарника.

Архитектура GPU определяется через nvidia-smi. При необходимости задавайте вручную:

```
make -C gpu double_fast -j GPU_ARCH=86 # Ampere
make -C gpu float_fast -j GPU_ARCH=89  # Ada
```

1.4 EPYC Zen5, AVX-512 и AVX2

Makefile берет CPU-флаги из `scripts/detect_arch_flags.sh` через `ARCH_FLAGS`. Для расчета на той же машине обычно лучше:

```
make -C cpu clean
make -C cpu -j ARCH_FLAGS="-march=native -mtune=native"

make -C gpu clean
make -C gpu double_fast -j ARCH_FLAGS="-march=native -mtune=native"
```

Цель	Флаги GCC/Clang	Комм
EPYC Zen2	ARCH_FLAGS="-march=znver2 -mtune=znver2"	AVX2
EPYC Zen3	ARCH_FLAGS="-march=znver3 -mtune=znver3"	AVX2
EPYC Zen4	ARCH_FLAGS="-march=znver4 -mtune=znver4"	AVX-5
EPYC Zen5	ARCH_FLAGS="-march=znver5 -mtune=znver5"	Нуже
Portable AVX2	ARCH_FLAGS="-O3 -mavx2 -mfma"	Запу
Явный AVX-512	ARCH_FLAGS="-O3 -mavx512f -mavx512dq -mavx512cd -mavx512bw -mavx512vl"	Толь

Флага AVX12 у GCC/Clang нет. Если имелся в виду Zen5 с AVX-512, используйте `-march=znver5`. Если компилятор старый, обновите его или задайте явные `-mavx512*` флаги после проверки `lscpu`.

2 Быстрый старт

```
# GPU double, полный диапазон 0..180 градусов
gpu/bin/mbs_po_gpu_double_fast --po -p 1 125.9 78.09 \
  --ri 1.3116 0 -w 0.532 -n 12 --oldauto 2 \
  --grid 0 180 600 180 --threads 64 --close -o hex_gpu_double

# То же, но CPU backend из GPU бинарника
gpu/bin/mbs_po_gpu_double_fast --po --cpu -p 1 125.9 78.09 \
  --ri 1.3116 0 -w 0.532 -n 12 --oldauto 2 \
  --grid 0 180 600 180 --threads 64 --close -o hex_cpu_from_gpu_bin

# Частица из файла, масштабирование через k_eq
gpu/bin/mbs_po_gpu_float_fast --po --pf particle.dat --k_eq 58.81 \
  --ri 1.6 0.002 -w 1.064 -n 14 --oldauto 2 --pole \
  --grid 0 180 600 180 --threads 16 --close -o particle_keq

# Быстрая проверка двух точек: 0 и 180 градусов
gpu/bin/mbs_po_gpu_double_fast --po -p 1 125.9 78.09 \
  --ri 1.3116 0 -w 0.532 -n 1 --oldauto 2 --pole \
  --grid 0 180 600 1 --threads 64 --close -o check_0_180
```

3 Физическая модель

3.1 Общий pipeline

Этап	Где в коде	Р
Геометрия частицы	src/particle, src/main.cpp	I
Трассировка лучей	src/scattering, src/tracer, src/Splitting.cpp	E
РО дифракция	HandlerP0::ApplyDiffraction*, Handler::DiffractIncline*, CUDA kernels	J
Когерентная сумма	HandlerP0::AddToMueller, CUDA fused Mueller kernels	C
Jones -> Mueller	src/math/Mueller.cpp	E
Усреднение ориентаций	HandlerP0Total, TracerP0Total	ф

3.2 Jones матрицы лучей

Каждый пучок несет комплексную матрицу Jones beam.J:

$$J = \begin{bmatrix} J_{vv} & J_{vh} \\ J_{hv} & J_{hh} \end{bmatrix}$$

При отражениях/преломлениях Splitting.cpp домножает beam.J на коэффициенты Fresnel в локальном вертикальном/горизонтальном базисе луча. В РО режиме это амплитуда, поэтому в обычном когерентном режиме сначала суммируются Jones матрицы пучков, и только затем считается Mueller.

3.3 Диады и поворот в базис рассеяния

Перед добавлением пучка в направление наблюдения локальный Jones базис надо перевести в базис детектора. Это делают HandlerPO::RotateJones и RotateJonesFast. Они строят 2x2 матрицу проекций через скалярные произведения между:

Вектор	Смысл
beam.Direction()	Направление пучка после трассировки
direction	Требуемое направление рассеяния
vf	Forward reference direction
info.polarization.vectors	Предвычисленный поляризационный базис пучка

В ApplyDiffractionFast вклад пучка имеет вид:

$$J_beam(\theta, \phi) = F_edge(\theta, \phi) * R_out(\theta, \phi) * F_n(\theta, \phi)$$

Множитель	Код	Смысл
F_edge	DiffractionInclineFast, DiffractionIncline, DiffractionInclineAbs	Скалярный edge-интеграл
R_out	RotateJones или KarczewskiJones	Поворот/проекция Jones базиса
F_n	ComputeFnJones	Fresnel/Jones поправка для пучка

Флаг --karczewski заменяет стандартный RotateJones на матрицу Karczewski. По комментариям в коде M11 при этом не меняется, потому что сохраняется Frobenius norm, но поляризационные элементы могут отличаться.

3.4 Дифракционный интеграл Kirchhoff

Для каждого выходного пучка освещенная апертура представлена полигоном. РО дальнейшее поле считается как интеграл по апертуре:

$$F(q) = \int_A \exp(i \mathbf{k} \cdot \mathbf{q} \cdot \mathbf{r}) dA$$

где A - полигон пучка, $k = 2\pi/\lambda$, q - разность между направлением пучка и направлением наблюдения. В коде интеграл считается через границы полигона, а не через sampling площади. Для ребра от a до b вклад выражается через разность комплексных экспонент и фазовый знаменатель; для малых знаменателей используются устойчивые sinc-пределы.

Функция	Назначение
Handler::DiffractionIncline	CPU scalar edge integral без поглощения
Handler::DiffractionInclineFast	Оптимизированный CPU edge integral с предвычисленными ребрами
Handler::DiffractionInclineAbs	Вариант с поглощением
GpuDiffraction.cu	CUDA реализация тех же вычислений

CPU и GPU ветки должны использовать одну геометрию, фазовую конвенцию и сетку theta. Отличия обычно дают точность (float/double), --use_fast_math, FFT-интерполяция, порядок редукции/атомики и специальные случаи полюсов/endpoint.

3.5 Когерентная и некогерентная сумма

По умолчанию:

```
J_total(theta, phi) = sum_beams J_beam(theta, phi)
M(theta, phi)       = Mueller(J_total(theta, phi))
```

С флагом `--incoh`:

```
M(theta, phi) = sum_beams Mueller(J_beam(theta, phi))
```

Преобразование Jones -> Mueller реализовано в `src/math/Mueller.cpp`: элементы 4x4 строятся из билинейных комбинаций $|S_i|^2, \text{Re}(S_i \text{ conj}(S_j)), \text{Im}(S_i \text{ conj}(S_j))$.

3.6 Усреднение ориентаций и `--pole`

Ориентационные режимы задают веса по β/γ . На точных β -полюсах все γ эквивалентны, поэтому `--pole` считает одну γ и умножает вес. В текущей версии при `--pole` используются β endpoints, чтобы точка $\beta=0$ или $\beta=\pi$ действительно присутствовала в сетке, а не заменялась `midpoint`.

3.7 Beca scattering grid

Для `uniform theta grid` выводится $N_{th} + 1$ строк. Колонка $2\pi \cdot d\cos$ - вес кольца телесного угла:

```
dOmega(theta_j) = 2*pi * (cos(theta_left) - cos(theta_right))
```

Для `endpoint` строк используется половинная ячейка, обрезанная границей диапазона. Для полной сетки $0 \dots 180$ сумма всех $2\pi \cdot d\cos$ должна быть 4π .

4 Частицы и размер

Флаг	Аргументы	Описание
-p	TYPE L D [extra]	Built-in частица. Нужно указать ровно один источник: -p или --pf.
--pf	FILE	Частица из файла.
--rs	SIZE	Масштабировать file particle до $D_{max} = \text{SIZE}$.
--k_eq	X	Масштабировать так, чтобы $k_{eq} = 2\pi \cdot r_{eq} / \lambda$.
--ri	Re Im	Комплексный показатель преломления.
-w	LAMBDA	Длина волны в микрометрах.
-n	N	Максимальная глубина внутренних отражений/преломлений.

Типы -p:

Type	Форма	Параметры
1	Hexagonal column/plate	L D
2	Bullet	L D
3	Bullet rosette	L D [cap]
4	Droxtal	L D extra
10	Concave hexagonal	L D concavity
12	Hexagonal aggregate	L D count
999	Built-in aggregate	extra

Масштабирование через `equivalent size`:

```
r_eq = (3 V / (4*pi))^(1/3)
k_eq = 2*pi*r_eq/lambda
scale = (k_eq_target * lambda / (2*pi)) / r_eq_original
```

5 Ориентационные сетки

Режим	Аргументы	Когда применять	Комментарий
--oldauto	DIV	Основной production режим	Шаг сетки связан с diffraction-limited
--random	Nb Ng	Ручная beta/gamma сетка	Использует symmetry-reduced domain
--fixed	BETA GAMMA	Отладка одной ориентации	Углы в градусах.
--orientfile	FILE	Пользовательские ориентации	Одна пара beta/gamma на строку.
--sobol	N	Quasi-random average	Хорош для сходимости сканов.
--sobol_seed	N S	Sobol/Owen с seed	Повторяемые проверки сходимости.
--sobol_ring	Nb Ng	Sobol beta + uniform gamma rings	Гибридная сетка.
--so3_quat	N	Full SO(3) quaternions	Не опирается на beta/gamma symmetry
--hammersley	N	Hammersley orientations	Experimental/debug.
--lattice	N	Rank-1 lattice	Experimental/debug.
--lattice_z	N Z	Rank-1 lattice с явным generator	Experimental/debug.
--euler_quad	Nb Ng	Gauss по cos(beta), periodic gamma	Высокий порядок квадратуры.
--euler_adapt	Nb NgMax	Adaptive gamma count	Меньше работы около beta poles.
--montecarlo	N	Псевдослучайные ориентации	Базовая Monte Carlo проверка.
--adaptive	EPS	Автоподбор числа Sobol orientations	Удваивает число ориентаций до целого
--auto	EPS	Auto theta, phi, orientations	Удобный режим.
--autofull	EPS	Auto n, theta, phi, orientations	Более полный и дорогой поиск.
--oldautofull	EPS	Autofull + oldauto final grid	Когда нужна регулярная финальная

Модификаторы:

Флаг	Аргументы	Описание
--ring_points	N	Точек на diffraction ring для oldauto estimates.
--mirror_gamma	none	Половина gamma domain + зеркалирование Mueller.
--sym	Sb Sg	Override symmetry: beta range π/Sb , gamma range $2\pi/Sg$.
--b	B1 B2	Диапазон beta для --random, градусы.
--g	G1 G2	Диапазон gamma для --random, градусы.
--maxorient	N	Верхняя граница adaptive orientations.
--chunk	N	Chunk size по ориентациям/gamma.
--coh_orient	none	Legacy coherent-across-orientations.
--pole	none	Одна gamma на точных beta poles.
--owen_avg	K	Усреднение K Owen seeds в --autofull.
--owen_seeds	S...	Явный список Owen seeds.

6 Сетка рассеяния

Флаг	Аргументы	Описание
--grid	T1 T2 Nphi Nth	Uniform theta range от T1 до T2; выводит Nth + 1 theta строк.
--grid	R Nphi Nth	Backscatter cone радиуса R градусов.
--tgrid	FILE	Неравномерная theta сетка, градусы, одна строка на theta.
--auto_tgrid	EPS	Adaptive theta grid через bisection.
--auto_phi	none	Автоматический выбор Nphi.
--nphi	N	Override phi count, максимальный приоритет.
--filter	DEG	Ограничить output backscatter cone.
--point	none	Legacy backscatter point mode.

Приоритет:

Величина	Приоритет
Theta	--tgrid > --grid > --auto_tgrid > --auto > default
Phi	--nphi > --grid > --auto_phi > --auto > default

7 GPU и multi-GPU

7.1 За счет чего GPU быстрее

Самая дорогая часть РО - многократный расчет дифракционного edge-интеграла для большого числа направлений, пучков и ориентаций. CPU готовит пучки, а GPU параллелит дифракцию и, в быстрых режимах, сразу накопление Mueller.

Операция	CPU path	GPU path
Трассировка через грани	OpenMP по ориентациям/gamma blocks	В основном CPU; --gpu_trace только CPU
Packing пучков	CPU	CPU готовит GpuBeam buffers и копирует
Edge diffraction	CPU loops	CUDA kernels
Когерентная Jones сумма	CPU complex arrays	Atomic или no-atomic device kernels
Jones -> Mueller	CPU после Jones sum	mueller_batch_kernel или fused kernel
Multi-k_eq	Повтор по размерам	Optional fused multi-k_eq CUDA pass
FFT phi	Direct phi grid	cuFFT low-phi pass + zero-padding

Масштаб работы:

$N_{work} \sim N_{orient} * N_{beams_per_orientation} * N_{theta} * N_{phi}$

Эти вклады почти независимы до момента когерентной суммы Jones, поэтому хорошо ложатся на CUDA.

7.2 Atomic и no-atomic режимы

Режим	Kernels
Atomic Jones	diffraction_kernel
No-atomic grid	diffraction_grid_kernel
Fused Mueller	diffraction_grid_mueller_kernel, *_full_kernel, *_full8_kernel, *_mixed
Staged orientation Mueller	diffraction_grid_mueller_orient_kernel + reduce_mueller_orient_kernel
Multi-k_eq fused	diffraction_grid_mueller_multik_kernel

В atomic path атомы идут по real/imag компонентам 2x2 Jones:

J00.re, J00.im, J01.re, J01.im,
J10.re, J10.im, J11.re, J11.im

Для _noshadow тот же набор atomics пишется во второй Jones buffer. Это корректно для когерентной РО суммы, но может быть узким местом, если много пучков пишут в одну detector cell.

Управление:

Переменная	Действие
MBS_GPU_NO_ATOMICS=1	Форсировать no-atomic путь, если поддерживается.
MBS_GPU_NO_ATOMICS=0	Форсировать atomic путь для сравнения/debug.
MBS_GPU_FUSED_MUELLER=1	Предпочитать fused diffraction-to-Mueller kernels.
MBS_GPU_STAGED_MUELLER=1	Включить staged per-orientation Mueller reduction.
MBS_GPU_TIMING=1	Печатать breakdown count/pack/copy/kernels/d2h/add.
MBS_GPU_BLOCK=N	Override CUDA block size.

7.3 Multi-size и multi-GPU

Флаг	Аргументы	Описание
--multigrid	Dmin Dmax N	Log-spaced scan по Dmax.
--multikeq	Kmin Kmax N	Log-spaced scan по k_eq.
--multikeq_list	FILE	Точный список k_eq, один на строку.
--multigrid_parallel	N	Запуск scan points как child processes; 0 значит auto.
--multigrid_threads	N	OpenMP threads на child process.
--gpu_devices	LIST	Список CUDA devices, например 0,1,2,3,4.
--multikeq_shared_batches	none	Batch близких k_eq на GPU child с reuse tracing от максимал
--multikeq_batch_ratio	R	Максимальное kmax/kmin внутри shared batch; default 1.05.

Exact multi-GPU:

```
gpu/bin/mbs_po_gpu_float_fast --po --pf particle.dat \
--multikeq_list keq.txt --ri 1.6 0.002 -w 1.064 -n 12 \
--oldauto 2 --grid 0 180 600 180 --fft --close \
--multigrid_parallel 0 --multigrid_threads 16 --gpu_devices 0,1,2,3,4 \
-o scan_exact
```

Shared-batch:

```
gpu/bin/mbs_po_gpu_float_fast --po --pf particle.dat \
--multikeq_list keq.txt --ri 1.6 0.002 -w 1.064 -n 12 \
--oldauto 2 --grid 0 180 600 180 --fft --close \
--multigrid_parallel 0 --multigrid_threads 16 --gpu_devices 0,1,2,3,4 \
--multikeq_shared_batches --multikeq_batch_ratio 1.05 \
-o scan_shared
```

Fused multi-k_eq:

```
MBS_GPU_MULTI_K_FULL=1 gpu/bin/mbs_po_gpu_float_fast ...
```

8 Все флаги

8.1 Метод, частица, физические параметры

Флаг	Аргументы	Описание
--po	none	Physical Optics.
--go	none	Geometrical Optics.
-p	TYPE L D [extra]	Built-in particle.
--pf	FILE	Particle from file.
--rs	SIZE	Resize file particle by Dmax.
--k_eq	X	Resize by equivalent size parameter.
--ri	Re Im	Refractive index.
-w	LAMBDA	Wavelength, micrometers.
-n	N	Max internal reflections/refractions.
--abs	none	Enable absorption accounting.
--abs_points	N или all	Absorption samples.

8.2 Ориентации

Флаг	Аргументы	Описание
--fixed	BETA GAMMA	Одна ориентация в градусах.
--random	Nb Ng	Regular beta/gamma grid.
--sobol	N	Sobol orientations.
--so3_quat	N	Full SO(3) quaternions.

Флаг	Аргументы	Описание
--sobol_seed	N S	Sobol with Owen seed.
--sobol_ring	Nb Ng	Sobol beta x gamma rings.
--hammersley	N	Hammersley set.
--lattice	N	Rank-1 lattice.
--lattice_z	N Z	Rank-1 lattice with generator.
--euler_quad	Nb Ng	Euler/Gauss quadrature.
--euler_adapt	Nb NgMax	Adaptive gamma count.
--montecarlo	N	Monte Carlo orientations.
--adaptive	EPS	Adaptive orientation convergence.
--auto	EPS	Auto theta, phi, orientation count.
--autofull	EPS	Auto n, theta, phi, orientation count.
--oldautofull	EPS	Autofull + oldauto final grid.
--owen_avg	K	Average final Owen seeds.
--owen_seeds	S...	Explicit Owen seeds.
--oldauto	DIV	Physics-based regular grid.
--ring_points	N	Points per diffraction ring estimate.
--mirror_gamma	none	Mirror half gamma domain.
--orientfile	FILE	Load beta/gamma from file.
--b	B1 B2	Beta range for --random.
--g	G1 G2	Gamma range for --random.
--maxorient	N	Max adaptive orientations.
--chunk	N	Orientation/gamma chunk size.
--coh_orient	none	Legacy coherent orientation mode.
--pole	none	Exact beta-pole gamma shortcut.
--sym	Sb Sg	Override symmetry.

8.3 Сетка, ускорение, cutoffs

Флаг	Аргументы	Описание
--grid	T1 T2 Nphi Nth	Uniform theta range.
--grid	R Nphi Nth	Backscatter cone.
--tgrid	FILE	Non-uniform theta grid.
--auto_tgrid	EPS	Adaptive theta grid.
--auto_phi	none	Auto phi count.
--nphi	N	Override phi count.
--threads	N	OpenMP worker threads.
--gpu	none	CUDA backend.
--cpu	none	Force CPU backend in GPU build.
--fft	none	cuFFT phi interpolation.
--beam_cutoff	EPS	Set beam J and area cutoffs.
--beam_cutoff_j	EPS	Skip by relative ‘
--beam_cutoff_area	EPS	Skip by relative area.
--beam_cutoff_importance	EPS	Skip by relative ‘
--trace_cutoff	EPS	Set trace pruning cutoffs.
--trace_cutoff_j	EPS	Trace prune by ‘
--trace_cutoff_area	EPS	Trace prune by area.
--trace_cutoff_importance	EPS	Trace prune by ‘
--trace_max_beams	N	Abort orientation after N beam nodes; 0 disables.
--gpu_trace	none	Experimental CUDA candidate prefilter.
--trace_prefilter	none	Enable CPU projected-AABB prefilter.
--no_trace_prefilter	none	Disable CPU prefilter.
--trace_prefilter_margin	M	AABB prefilter margin.
--trace_prefilter_stats	none	Print prefilter counters.
-r	RATIO	Beam area restriction ratio.

8.4 Output, diagnostics, legacy

Флаг	Аргументы	Описание
-o	NAME	Output path/name.
--close	none	Exit after computation.
--log	SEC	Progress log interval.
--checkpoint	none	Save/resume long runs.
--save_betas	none	Save per-beta Mueller files.
--full_only	none	Write only full Mueller output; default.
--noshadow_output	none	Also write _noshadow.
--shadow	none	Legacy flag, currently no effect.
--shadow_off	none	Disable shadow beam.
--incoh	none	Incoherent per-beam Mueller sum.
--jones	none	Write Jones matrices where supported.
--karczewski	none	Use Karczewski polarization matrix.
--legacy_sign	none	Old Fresnel sign convention.
--ot_phase_avg	none	Average OT extinction over phase period.
--ot_phase_shift	F	Diagnostic OT phase shift.
--ot_ping	D	Legacy OT phase rotation.
--filter	DEG	Restrict output to backscatter cone.
--point	none	Legacy backscatter point mode.
--tr	FILE	Load trajectory file.
--all	none	Calculate all loaded trajectories.
--gr	none	Output trajectory groups.
--forced_convex	none	Force convex processing.
--forced_nonconvex	none	Force nonconvex processing.
--help, -h	none	Short help.
--help-debug	none	Full debug/experimental help.

9 Переменные окружения

Переменная	СМЫСЛ
CUDA_VISIBLE_DEVICES	Стандартный выбор видимых CUDA devices.
OMP_NUM_THREADS	OpenMP threads по умолчанию.
MBS_GPU_ALLOW_FALLBACK=1	Разрешить старый fallback GPU->CPU после CUDA ошибки. Только deb
MBS_GPU_MEM_FRACTION	Доля свободной GPU памяти для buffers.
MBS_GPU_NO_ATOMICS	Выбор no-atomic (1) или atomic (0) path.
MBS_GPU_FUSED_MUELLER	Fused diffraction-to-Mueller kernels.
MBS_GPU_STAGE_MUELLER	Staged per-orientation Mueller reduction.
MBS_GPU_NO_VERTEX_CACHE	Отключить cached/packed vertex path.
MBS_GPU_TIMING	Печатать CUDA timing breakdown.
MBS_GPU_BLOCK	Override CUDA block size.
MBS_HOST_MEM_FRACTION	Доля host RAM для oldauto/random chunking.
MBS_HOST_MEM_RESERVE_MB	Резерв host RAM в MB.
MBS_HOST_MEM_BUDGET_MB	Жесткий host RAM budget.
MBS_FFT_PHI_FACTOR	Override reduced phi factor для FFT.
MBS_GPU_MULTI=0	Отключить automatic multi-orientation GPU batching.
MBS_GPU_MULTI_MAX=N	Ограничить GPU multi batching.
MBS_GPU_MULTI_K_FULL=1	Experimental fused multi-k_eq diffraction.
MBS_SHARED_BETA_GROUP=N	Override beta grouping для shared batch.
MBS_SHARED_ORIENT_CHUNK=N	Override orientation chunk для shared batch.
MBS_PARALLEL_MEM_FRACTION	Memory fraction scheduler для parallel multi-size.
MBS_PARALLEL_SHARED_KEQ=1	Environment equivalent для shared k_eq batching.
MBS_PARALLEL_KEQ_BATCH_RATIO=R	Default batch ratio для shared k_eq.

10 Выходные файлы

Основной .dat:

ScAngle 2pi*dcos M11 M12 M13 M14 M21 M22 M23 M24 M31 M32 M33 M34 M41 M42 M43 M44

Колонка	Смысл
ScAngle	Угол рассеяния theta в градусах.
2pi*dcos	Вес кольца телесного угла для theta row.
Mij	Элементы Mueller после суммирования пучков и усреднения ориентаций.

Дополнительные outputs:

Output	Как включается	Смысл
_noshadow	--noshadow_output	Mueller без shadow/external beam.
_betas/	--save_betas	Per-beta diagnostics.
checkpoint files	--checkpoint	Resume для длинных orientation-grid runs.
Jones output	--jones	Raw Jones matrices в поддержанных PO modes.

Предупреждения про optical-theorem/integral mismatch являются диагностикой. Для non-absorbing runs физическое поглощение фиксируется нулем; integral characteristic не заменяет выходную Mueller матрицу.